Rapport Projet “Application VBA & Apps Script Évènements du département SD”

Répartition des tâches au sein du groupe :

Théo :

Dans ce projet, je me suis chargé de l’ensemble du développement réalisé sur **Excel et en VBA.** J’ai conçu l’architecture des feuilles, leur organisation fonctionnelle ainsi que les automatisations nécessaires au bon fonctionnement de l’outil. J’ai réalisé cette partie intégralement de A à Z, avec pour objectif de proposer une interface claire, ergonomique et suffisamment robuste pour être utilisée comme une véritable application de gestion.

L’outil Excel repose principalement sur trois feuilles :

- *Menu*,
- *Étudiant*
- *Tableau de bord (TDB)*,

complétées par des feuilles techniques destinées aux calculs intermédiaires et aux filtrages (**masquées**). La feuille *Menu* constitue le point d’entrée principal dans l’application. Elle permet à l’utilisateur de naviguer vers les différentes fonctionnalités à l’aide de boutons visuels donnant accès au formulaire d’ajout d’un étudiant, à la base des étudiants et au tableau de bord. J’ai également mis en place un système permettant de basculer l’interface en mode plein écran ou en mode réduit afin de proposer un affichage plus immersif si nécessaire. Cette feuille contient volontairement très peu de VBA, en dehors des macros liées à la gestion de l’affichage, afin de conserver une interface simple, rapide et intuitive.

La feuille *Étudiant* constitue la base de données opérationnelle de l’outil. Elle regroupe l’ensemble des dossiers enregistrés via le formulaire. Certaines colonnes sont automatiquement calculées et apparaissent en gris afin de les distinguer des champs saisis manuellement, no-

tamment l'identifiant étudiant, l'âge (calculé à partir de la date de naissance) et la date d'insertion du dossier. J'ai automatisé la génération des identifiants afin de garantir leur unicité et d'éviter toute erreur de saisie. De la même manière, la date d'insertion n'est renseignée que lorsque l'ensemble des champs obligatoires est correctement rempli, ce qui permet d'assurer la cohérence des données.

J'ai mis en place un système de liens dynamiques dans la colonne des identifiants. Cliquer sur un identifiant permet d'ouvrir le formulaire correspondant à l'étudiant sélectionné, avec ses informations pré-remplies, afin de pouvoir les modifier puis valider les changements. Une mise en forme automatique est également appliquée afin d'améliorer la lisibilité : ajustement dynamique des colonnes, homogénéisation du style et gestion spécifique des hyperliens, notamment pour les adresses électroniques.

La feuille *Tableau de bord* est dédiée à l'analyse et à la visualisation des données. J'y ai intégré plusieurs indicateurs clés de performance **(KPI's)** tels que le nombre de répondants, l'âge moyen ou le taux de satisfaction, ainsi que différents graphiques permettant de représenter visuellement la répartition des étudiants selon plusieurs critères, comme les tranches d'âge ou les canaux de découverte. J'ai également développé un système de sélection d'événements qui permet de filtrer dynamiquement l'ensemble des statistiques. Lorsqu'un événement est sélectionné, seules les données correspondantes sont prises en compte dans les calculs et les graphiques. Ce filtrage repose sur une feuille intermédiaire qui extrait automatiquement les étudiants liés à l'événement sélectionné.

Des boutons interactifs permettent d'afficher ou de masquer certains blocs[1] du tableau de bord. J'ai regroupé ces éléments et les ai contrôlés via VBA afin de permettre à l'utilisateur d'alléger l'interface ou de se concentrer sur certains indicateurs selon ses besoins, ce qui renforce le caractère dynamique et personnalisable de l'outil.

Sur le plan technique, j'ai fait le choix de réaliser un maximum de calculs directement avec les fonctions Excel afin de garantir la transparence des traitements, leur performance et leur stabilité. J'utilise le VBA principalement pour automatiser les tâches répétitives, gérer les interactions utilisateur, piloter l'affichage et assurer la cohérence des données. Les feuilles techniques me permettent de séparer clairement les données visibles des données de calcul, ce qui facilite la maintenance et l'évolution de l'outil.

Concernant l'utilisation de l'intelligence artificielle, je l'estime à **environ 60 % sur la partie VBA uniquement**. Je l'ai utilisée comme un support pour la génération de structures de code, la recherche de syntaxe ou de bonnes pratiques, ainsi que pour accélérer certains développements répétitifs. En revanche, l'architecture générale du fichier, l'organisation des feuilles, les choix fonctionnels, la logique métier et l'expérience utilisateur ont été entièrement conçus et implémentés manuellement.

Gabriel :

Gabriel a développé l'architecture de l'outil sur Google Sheets et Apps Script. Le système repose sur le fichier **Code.gs**, qui assure le bon fonctionnement de l'ensemble et fait le lien entre les feuilles de données ("Home" et "Inscrits") et les interfaces utilisateurs.

L'utilisation commence sur la feuille "Home". Le bouton "Débuter" déclenche l'ouverture du menu principal (fichier **Menu.html**), qui permet de naviguer vers les trois fonctionnalités clés :

- **L'ajout de dossier (Formulaire.html) :** Cette interface est dédiée à la saisie administrative. Elle présente 12 champs pour renseigner les informations personnelles et le statut de l'inscription. Une fois le formulaire rempli et enregistré, les données sont automatiquement enregistrées dans la base, c'est à dire insérées dans la feuille "Inscrits".

- **La gestion des inscrits (ListeInscrits.html) :** Ce module permet de rechercher un élève pour consulter, modifier ou supprimer son dossier. Il permet également d'exporter la fiche de l'élève au format PDF : pour cela, le programme utilise le fichier **ModelePDF.html** qui sert de modèle de mise en page.

- **Le tableau de bord (Dashboard.html) :** Cette page est dédiée à l'analyse visuelle. Elle affiche des graphiques dynamiques qui s'actualisent instantanément lorsque l'utilisateur applique des filtres sur la région, le sexe ou le diplôme.

Bilan technique et fonctionnel

Ce qui fonctionne L'objectif principal est atteint : l'ensemble des modules développés est pleinement opérationnel. La chaîne de traitement des données, depuis la saisie via le formulaire (Formulaire.html) jusqu'à l'archivage dans la base de données, s'exécute

sans erreur. De même, les fonctionnalités avancées comme la génération dynamique du PDF et la mise à jour en temps réel des graphiques du tableau de bord répondent parfaitement au cahier des charges initial. L'outil est fonctionnel et stable.

Les limites rencontrées (Ce qui ne fonctionne pas idéalement)
Le principal point faible réside dans la fluidité de la navigation. L'utilisateur peut ressentir une certaine latence lors du passage d'une interface à l'autre. Deux facteurs expliquent ce phénomène :

1. **L'optimisation des scripts :** Certains modules sont relativement lourds en traitement. Le chargement des interfaces HTML et l'appel aux données nécessiteraient une refactorisation du code pour alléger les processus et accélérer l'affichage.

2. **Les contraintes de l'environnement Web :** Contrairement à un logiciel de bureau comme Excel qui utilise les ressources locales de l'ordinateur, Google Sheets est une solution "Cloud". La réactivité de l'outil est donc tributaire de la qualité de la connexion internet et des délais de réponse des serveurs Google (temps d'exécution des scripts Apps Script), ce qui rend l'expérience moins fluide qu'une application native.

Perspectives et propositions d'évolution Pour aller plus loin et enrichir l'outil, plusieurs fonctionnalités pourraient être implémentées dans une version ultérieure :

- **Automatisation des emails :** Actuellement, le PDF est généré et stocké. Il serait pertinent d'ajouter une fonction d'envoi automatique du PDF par email à l'étudiant dès la validation de son inscription.

- **Gestion des droits d'utilisateurs :** Créer un système d'authentification pour distinguer les administrateurs (qui peuvent tout supprimer/modifier) des utilisateurs simples (qui peuvent seulement consulter ou saisir), afin de sécuriser les données.

Concernant le développement Apps Script géré par Gabriel, le taux d'utilisation de l'IA est estimé à environ **75%**. Bien que Gabriel apprécie les langages utilisés (HTML, CSS et JavaScript), il a choisi d'utiliser l'IA comme un accélérateur de production. Elle a servi à générer rapidement la structure brute des fichiers et le code répétitif, permettant à Gabriel de se concentrer sur l'architecture logique, l'assemblage des différents modules et la correction des bugs, mais aussi à l’expérience utilisateur.